

Exportación a Excel y PDF de la Liquidación de Mercancía

Documento técnico de implementación — Odoo 19

Modelo objetivo	stock.landed.cost.run (Liquidación / Landed Cost Run)
Vista objetivo	stock_landed_costs_features.stock_landed_cost_run_form
Módulos involucrados hoy	stock_landed_costs_features (v19.0.1.0.5), stock_landed_costs_file (v19.0.1.0.3)
Entregables	Reporte XLSX multi-hoja (espejo de la pantalla) + reporte QWeb PDF apaisado
Versión destino	Odoo 19.0

1. Objetivo

Permitir que un usuario, parado en el formulario de una liquidación de mercancía, descargue:

1. Un **archivo Excel (.xlsx)** que reproduzca la pantalla completa: cabecera, las cuatro pestañas (Facturas de proveedor, Productos pre-despacho, Costos en destino, Resumen de valoración) y los totales del pie.
2. Un **PDF** legible y paginado correctamente, en sustitución de la impresión del navegador que hoy sale mal.

Todos los campos visibles de la pantalla —incluidos los que están como columnas opcionales ocultas— entran en el alcance.

2. Diagnóstico: por qué hoy no se puede

2.1 Excel

El exportador nativo de Odoo (Acciones ▶ Exportar) **solo existe en vistas lista/kanban que son la vista raíz de una acción de ventana**. La liquidación muestra sus cuatro bloques como campos 0ne2many embebidos dentro de un <form> :

```

stock.landed.cost.run (form)
├─ vendor_bill_ids    → list embebida  (account.move)
├─ product_ids        → list embebida  (stock.landed.cost.product)
├─ landed_cost_ids    → list embebida  (stock.landed.cost)
└─ summary_ids        → list embebida  (stock.valuation.adjustment.summary)

```

Una lista embebida en un formulario no renderiza el menú Acciones , por lo que no ofrece «Exportar» ni «Insertar en hoja de cálculo». Es una limitación del cliente web, no de permisos.

A esto se suman dos detalles del modelo:

- fob_currency , additional_cost_total_currency y amount_total_currency son campos **calculados no almacenados** (compute="_compute_amount_all" , sin store=True). Aunque la lista raíz de liquidaciones sí exporta, estos totales no son agrupables ni filtrables, y el exportador nativo los trae por lectura fila a fila.
- La única lista del expediente que hoy sí es exportable de forma nativa es stock.valuation.adjustment.summary , porque tiene acción propia (stock_valuation_adjustment_summary_action) accesible desde el botón inteligente de la ficha de producto. Es un acceso indirecto, sin la cabecera ni los totales de la liquidación.

2.2 PDF

Ni stock_landed_costs (core) ni stock_landed_costs_features declaran ningún ir.actions.report sobre stock.landed.cost.run . El botón «Imprimir» del menú de Odoon no ofrece nada, de modo que el usuario recurre a Ctrl+P del navegador, que imprime el DOM del backend. Eso produce:

- **Solo la pestaña activa** del notebook se imprime; las otras tres están en el DOM pero ocultas por CSS.
- Las listas con scroll horizontal se cortan en el ancho de página.
- Las columnas marcadas optional="hide" (quantity , weight , volume , difference_move_id , preclearance_product_id) no aparecen.
- El widget OWL cost_factor_indicator_widget no tiene representación imprimible.
- Se arrastran barras de herramientas, breadcrumb, chatter y botones.

La conclusión es que ambos entregables requieren desarrollo; no hay configuración que los resuelva.

3. Alternativas evaluadas y descartadas

Alternativa	Por qué se descarta
Publicar acciones de ventana y menús para cada modelo hijo, y usar el exportador nativo	Genera cuatro archivos distintos, sin cabecera, sin totales, sin formato, y obliga al usuario a filtrar por liquidación en cada uno. No es «la pantalla en Excel».
«Insertar en hoja de cálculo» (Documents Spreadsheet, Enterprise)	Mismo bloqueo: solo disponible en vistas lista/pivote raíz. Tampoco reproduce la cabecera ni el pie.

Alternativa	Por qué se descarta
Sacar los datos por un informe <code>account.report</code> o SQL view	Serviría para análisis transversal, no para el documento de archivo del expediente que pide el cliente.
Imprimir con <code>report_py3o</code> sobre plantilla ODS/XLSX	Añade dependencia de LibreOffice en el contenedor. <code>report_xlsx</code> es más liviano y ya está en uso en el repositorio.

4. Dónde va el desarrollo: módulo aparte

Recomendación: módulo nuevo `stock_landed_costs_run_report` .

4.1 Razones

1. **Dependencia OCA.** El reporte XLSX se apoya en `report_xlsx` (OCA `reporting-engine` , v19.0.1.0.2, ya presente en el repositorio) y en la librería Python `xlsxwriter` . Meter esa dependencia dentro de `stock_landed_costs_features` obliga a que **todo** despliegue que hoy usa ese módulo incorpore el submódulo `OCA/reporting-engine` y la librería, aunque no quiera el reporte.
2. **`stock_landed_costs_features` es un módulo base compartido.** Ya tiene otro módulo encima (`stock_landed_costs_file`) y una suite de tests propia. Un cambio de presentación no debe forzar subida de versión ni re-validación funcional del módulo que hace los cálculos de valoración.
3. **Adopción opt-in.** Cada cliente decide si instala el reporte. Si mañana otro cliente quiere otro formato, se hace un segundo módulo satélite sin tocar la base.
4. **Ciclo de vida distinto.** Las plantillas de impresión cambian por pedido del cliente (columnas, logo, orden); la lógica de costos no.

4.2 Alternativa: todo dentro de `stock_landed_costs_features`

Es viable técnicamente: se agrega `report_xlsx` a `depends` , un directorio `report/` y se sube la versión a `19.0.1.1.0` . El costo es que los cuatro puntos anteriores se convierten en obligaciones para todos los despliegues existentes. Solo se justifica si la política del proyecto es no multiplicar módulos.

4.3 Dependencias del módulo nuevo

```
"depends": ["stock_landed_costs_file", "report_xlsx"],
```

Se depende de `stock_landed_costs_file` (y no solo de `stock_landed_costs_features`) por dos motivos:

- El campo **Expedientes** (`lc_file_ids`) que va en la hoja de cabecera lo define ese módulo.
- Existe un acoplamiento ya presente en el código:
`stock_landed_costs_features/models/stock_landed_cost_run.py` referencia `self.lc_file_ids` en `_check_lc_file_ids()` y en `button_generate_landed_costs()` , pero el campo se declara en `stock_landed_costs_file` . En la práctica, `features` no opera sin `file` .

Si se quisiera soportar instalaciones sin expediente, se depende solo de `stock_landed_costs_features` y se lee el campo de forma defensiva:

```
files = run.lc_file_ids if "lc_file_ids" in run._fields else run.browse()
```

5. Inventario de campos por hoja

Este es el contrato del reporte. Cada hoja del libro corresponde a un bloque de la pantalla.

Hoja 1 — «Liquidación» (cabecera + totales)

Campo	Etiqueta	Tipo
name	Liquidación	Char
number	No. B/L	Char
date_bl	Fecha B/L	Date
manifest_ref	Manifiesto	Char
date	Fecha	Date
state	Estado	Selection (Borrador/Validado/Cancelado)
company_id	Compañía	Many2one
lc_file_ids	Expedientes	Many2many (texto separado por comas)
currency_id	Moneda	Many2one (informativo; moneda de la compañía)
fob_currency	Monto FOB	Monetary (calculado, no almacenado)
additional_cost_total_currency	Total costos adicionales	Monetary (calculado, no almacenado)
amount_total_currency	Costo total	Monetary (calculado, no almacenado)
total_cost_factor_currency	Factor de costo promedio	Float, precisión «Cost Factor» (4 decimales)

Hoja 2 — «Facturas de proveedor» (`vendor_bill_ids` → `account.move`)

Campo	Etiqueta	Notas
name	Factura	

Campo	Etiqueta	Notas
partner_id	Proveedor	
invoice_date	Fecha de factura	
amount_untaxed_signed	Subtotal	Monetary, con fila de suma
amount_total_signed	Total	Monetary, con fila de suma
state	Estado	

Visible en la vista solo para `account.group_account_invoice` .

Hoja 3 — «Productos pre-despacho» (`product_ids` → `stock.landed.cost.product`)

Campo	Etiqueta	Notas
product_id	Producto	
name	Etiqueta	
account_id	Cuenta	
quantity	Cantidad	
product_uom_id	Unidad de medida	Solo si <code>uom.group_uom</code>
price_unit	Precio unitario	Precisión «Product Price»
price_subtotal	Subtotal	Monetary, con fila de suma
general_split_method	Método de reparto por defecto	Selection
invoice_id	Factura	
difference_move_id	Asiento de diferencia	Columna opcional en la vista; se incluye
clearance_has_difference	¿Tiene diferencia?	Invisible en la vista; se incluye como Sí/No

Página visible solo para `stock.landed_costs.features.group_preclearance` .

Hoja 4 — «Costos en destino» (`landed_cost_ids` → `stock.landed.cost`)

Campo	Etiqueta	Notas
name	Costo en destino	
picking_ids	Recepciones	Lista de referencias separadas por coma
date	Fecha	

Campo	Etiqueta	Notas
vendor_bill_id	Factura de proveedor	
preclearance_product_id	Producto de costo	Opcional en vista; grupo pre-despacho
account_journal_id	Diario	
state	Estado	
amount_total	Total	Monetary, con fila de suma

Hoja 5 — «Resumen de valoración» (summary_ids → stock.valuation.adjustment.summary)

Campo	Etiqueta	Notas
product_id	Producto	
former_cost	Costo anterior	Monetary, unitario
additional_landed_cost	Costo adicional	Monetary, unitario
final_cost	Costo final	Monetary, unitario
split_method	Método de reparto	Selection
cost_factor_currency	Factor de costo	Float 4 decimales
previous_cost_factor_currency	Factor anterior	Reemplaza al widget OWL
(derivado)	Variación	cost_factor_currency – previous_cost_factor_currency , con formato condicional verde/rojo
quantity	Cantidad comprada	Opcional en vista
weight	Peso	Opcional en vista
volume	Volumen	Opcional en vista

Pie de hoja con los cuatro totales de la liquidación, replicando el oe_subtotal_footer de la pantalla.

Anexo opcional — El formulario tiene un botón inteligente «Detalles de valoración» que abre `stock.valuation.adjustment.lines`. No es parte de la pantalla solicitada, pero puede añadirse como sexta hoja con el detalle línea a línea (producto, movimiento, línea de costo, costo anterior, costo adicional) sin cambiar la arquitectura.

6. Diseño del reporte XLSX

6.1 Definición de la acción

```
<record id="action_report_landed_cost_run_xlsx" model="ir.actions.report">
    <field name="name">Liquidación (XLSX)</field>
    <field name="model">stock.landed.cost.run</field>
    <field name="report_type">xlsx</field>
    <field name="report_name">stock_landed_costs_run_report.landed_cost_run_xlsx</field>
    <field name="report_file">stock_landed_costs_run_report.landed_cost_run_xlsx</field>
    <field name="binding_model_id" ref="stock_landed_costs_features.model_stock_landed_cost_run"/>
    <field name="binding_type">report</field>
    <field name="print_report_name">'%s.xlsx' % (object.name)</field>
</record>
```

`binding_model_id + binding_type="report"` hace que aparezca en el menú **Imprimir** tanto del formulario como de la lista de liquidaciones (selección múltiple incluida).

6.2 Modelo abstracto del reporte

```
from odoo import _, models
```

```
class LandedCostRunXlsx(models.AbstractModel):
    _name = "report.stock_landed_costs_run_report.landed_cost_run_xlsx"
    _inherit = "report.report_xlsx.abstract"
    _description = "Landed Cost Run XLSX Report"

    def generate_xlsx_report(self, workbook, data, runs):
        formats = self._get_formats(workbook)
        multi = len(runs) > 1
        for run in runs:
            self._create_header_sheet(workbook, formats, run, multi)
            if self.env.user.has_group("account.group_account_invoice"):
                self._create_vendor_bill_sheet(workbook, formats, run, multi)
            if self.env.user.has_group("stock_landed_costs_features.group_preclearance"):
                self._create_preclearance_sheet(workbook, formats, run, multi)
            self._create_landed_cost_sheet(workbook, formats, run, multi)
            self._create_summary_sheet(workbook, formats, run, multi)
```

Puntos de diseño:

- **Una hoja por pestaña**, siempre en el mismo orden que la pantalla.
- **Selección múltiple**: si se marcan varias liquidaciones en la lista, el nombre de la hoja se prefija con `run.name`. `report_xlsx` ya parchea `xlsxwriter.Workbook._check_sheetname` para deduplicar y respetar el límite de 31 caracteres.
- **Los grupos de la vista se replican en Python**. Un reporte no aplica los `groups=` del XML; hay que comprobarlos explícitamente con `has_group`, si no un usuario sin permiso de facturación vería las facturas de proveedor en el Excel.
- Las reglas de registro (`ir.rule`) y los ACL sí se aplican solos, porque el reporte lee con el entorno del usuario.

6.3 Formatos de celda

```
def _get_formats(self, workbook):
    currency = self.env.company.currency_id
    money = self._report_xlsx_currency_format(currency) # helper de report_xlsx
    lang = self.env["res.lang"]._lang_get(self.env.user.lang)
    date_fmt = (
        lang.date_format.replace("%d", "dd").replace("%m", "mm").replace("%Y", "yyyy")
    )
    return {
        "title": workbook.add_format({"bold": 1, "font_size": 14}),
        "label": workbook.add_format({"bold": 1, "align": "right"}),
        "header": workbook.add_format(
            {"bold": 1, "border": 1, "align": "center", "fg_color": "#F2F2F2", "text_wrap": 1}
        ),
        "text": workbook.add_format({"border": 1}),
        "date": workbook.add_format({"border": 1, "num_format": date_fmt}),
        "money": workbook.add_format({"border": 1, "num_format": money}),
        "money_total": workbook.add_format(
            {"bold": 1, "top": 2, "border": 1, "num_format": money}
        ),
        "factor": workbook.add_format({"border": 1, "num_format": "0.0000"}),
        "factor_up": workbook.add_format({"border": 1, "num_format": "0.0000", "font_color": "#C00000", "text_wrap": 1}),
        "factor_down": workbook.add_format({"border": 1, "num_format": "0.0000", "font_color": "#008000", "text_wrap": 1})
    }
```

- La máscara monetaria sale de `_report_xlsx_currency_format()`, que ya resuelve símbolo, posición y decimales de la moneda.
- La máscara de fecha se deriva de `res.lang` del usuario, no se codifica.
- El factor de costo usa 4 decimales, alineado con la precisión decimal `Cost Factor` que define `stock_landed_costs_features/data/decimal_precision.xml`.

6.4 Comportamiento de cada hoja

- **Fila 1**: título del bloque + nombre de la liquidación.

- **Fila de encabezados** con formato `header` , `freeze_panes` justo debajo y `autofilter` sobre el rango de datos.
- **Anchos de columna** fijados por `set_column` según el tipo (texto largo 40, fechas 14, importes 16).
- **Fila de totales** al cierre de cada hoja que lleve sumas en la pantalla (`price_subtotal` , `amount_total` , `amount_untaxed_signed` , `amount_total_signed`), escrita como fórmula `=SUM(...)` para que el usuario pueda filtrar y recalcular.
- **Números como números**, nunca como texto: se escribe el valor flotante y el formato se aplica por `num_format` . Esto es lo que permite pivotear el archivo después.
- `sheet.repeat_rows(header_row)` y `sheet.fit_to_pages(1, 0)` para que la hoja también imprima bien desde Excel.
- En la hoja de resumen, la columna **Variación** se escribe con `factor_up` / `factor_down` según el signo, reproduciendo la semántica de `cost_factor_indicator_is_positive` (el indicador es «positivo» cuando el factor actual es **menor** que el anterior).

6.5 Valores calculados no almacenados

`fob_currency` , `additional_cost_total_currency` y `amount_total_currency` se leen directamente del `recordset`; el `compute` se dispara al acceder. No requieren tratamiento especial, pero **no pueden usarse en `read_group` ni en dominios**, por lo que cualquier futura variante agregada del reporte debe recalcularlos en Python.

7. Diseño del reporte PDF (QWeb)

7.1 Formato de papel

Cinco bloques con hasta once columnas no caben en A4 vertical. Se declara un formato propio:

```
<record id="paperformat_landed_cost_run" model="report.paperformat">
  <field name="name">Liquidación de mercancía (apaisado)</field>
  <field name="format">A4</field>
  <field name="orientation">Landscape</field>
  <field name="margin_top">32</field>
  <field name="margin_bottom">20</field>
  <field name="margin_left">6</field>
  <field name="margin_right">6</field>
  <field name="header_spacing">28</field>
  <field name="dpi">90</field>
</record>
```

7.2 Acción de reporte

```
<record id="action_report_landed_cost_run" model="ir.actions.report">
  <field name="name">Liquidación</field>
  <field name="model">stock.landed.cost.run</field>
  <field name="report_type">qweb-pdf</field>
  <field name="report_name">stock_landed_costs_run_report.report_landed_cost_run</field>
  <field name="report_file">stock_landed_costs_run_report.report_landed_cost_run</field>
  <field name="paperformat_id" ref="paperformat_landed_cost_run"/>
  <field name="binding_model_id" ref="stock_landed_costs_features.model_stock_landed_cost_run"/>
  <field name="binding_type">report</field>
  <field name="print_report_name">'%s' % (object.name)</field>
</record>
```

7.3 Estructura de la plantilla

```
<template id="report_landed_cost_run">
  <t t-call="web.html_container">
    <t t-foreach="docs" t-as="doc">
      <t t-call="web.external_layout">
        <div class="page o_landed_cost_run">
          <h2><span t-field="doc.name"/></h2>

          <!-- cabecera en dos columnas -->
          <div class="row mt-3">
            <div class="col-6">...B/L, Fecha B/L, Manifiesto, Expedientes...</div>
            <div class="col-6">...Fecha, Compañía, Estado...</div>
          </div>

          <!-- un bloque por pestaña -->
          <t t-call="stock_landed_costs_run_report.section_vendor_bills"/>
          <t t-call="stock_landed_costs_run_report.section_preclearance"/>
          <t t-call="stock_landed_costs_run_report.section_landed_costs"/>
          <t t-call="stock_landed_costs_run_report.section_summary"/>

          <!-- totales -->
          <div class="clearfix mt-4">...FOB, adicionales, total, factor promedio...</div>
        </div>
      </t>
    </t>
  </t>
</template>
```

Reglas de maquetación:

- Cada sección es una sub-plantilla independiente, para poder ocultarla por grupo
(groups="account.group_account_invoice" /

groups="stock_landed_costs_features.group_preclearance") y para que el cliente pueda pedir cambios sobre una sola sin tocar el resto.

- Las tablas usan <thead> real, para que wkhtmltopdf repita los encabezados al saltar de página.
- Cada bloque va envuelto en page-break-inside: avoid cuando es corto, y se deja romper cuando es largo.
- Importes con t-options="{ 'widget': 'monetary', 'display_currency': doc.currency_id }" ; factor con t-options="{ 'widget': 'float', 'precision': 4 }" .
- Si una sección no tiene líneas, se omite entera (nada de tablas vacías).

7.4 Hoja de estilos

```
.o_landed_cost_run {  
  table { font-size: 9px; }  
  th { background-color: #f2f2f2; }  
  td.text-end { white-space: nowrap; }  
}
```

Se registra en web.report_assets_common .

8. Estructura del módulo

```
stock_landed_costs_run_report/  
├── __init__.py  
├── __manifest__.py  
├── README.rst  
├── i18n/  
│   └── es_D0.po  
├── data/  
│   └── report_paperformat.xml  
├── report/  
│   ├── __init__.py  
│   ├── landed_cost_run_xlsx.py          # AbstractModel report....xlsx  
│   ├── landed_cost_run_xlsx.xml        # ir.actions.report (xlsx)  
│   ├── landed_cost_run_report.xml      # ir.actions.report (qweb-pdf)  
│   └── landed_cost_run_templates.xml    # plantillas QWeb + secciones  
├── static/src/scss/  
│   └── landed_cost_run_report.scss  
└── tests/  
    ├── __init__.py  
    └── test_landed_cost_run_report.py
```

Manifiesto:

```

{
  "name": "Landed Cost Run Reports",
  "summary": "XLSX and PDF reports for landed cost runs",
  "author": "INDEXA SRL.",
  "website": "https://www.progressa.group/",
  "license": "Other proprietary",
  "category": "Stock",
  "version": "19.0.1.0.0",
  "depends": ["stock_landed_costs_file", "report_xlsx"],
  "data": [
    "data/report_paperformat.xml",
    "report/landed_cost_run_templates.xml",
    "report/landed_cost_run_report.xml",
    "report/landed_cost_run_xlsx.xml",
  ],
  "assets": {
    "web.report_assets_common": [
      "stock_landed_costs_run_report/static/src/scss/landed_cost_run_report.scss",
    ],
  },
  "installable": True,
}

```

9. Seguridad y multicompañía

- **No se crean modelos nuevos**, por lo tanto no hay ACL ni reglas de registro nuevas. El reporte lee con el entorno del usuario y hereda los `ir.model.access` de `stock_landed_costs_features` (`stock.group_stock_manager` sobre `stock.landed.cost.run` , `stock.landed.cost.product` y `stock.valuation.adjustment.summary`).
- **Los groups= de la vista deben re-implementarse en el generador** (ver 6.2). Es el punto de seguridad más fácil de olvidar: un reporte no respeta los grupos declarados en el XML del formulario.
- **Exportación:** el reporte no pasa por `base.group_allow_export` , por lo que funciona incluso para usuarios a los que se les restringió la exportación nativa. Si el cliente quiere mantener esa restricción, hay que añadir `groups_id` a la `ir.actions.report` . Es una decisión funcional a confirmar.
- **Moneda:** todos los importes de la liquidación están en moneda de la compañía (`currency_id` es `related` de `company_id.currency_id`). No hay conversión que hacer, pero el símbolo del formato se toma de la compañía de la liquidación, no de la del usuario.

10. Internacionalización

Las etiquetas del XLSX se escriben desde Python y deben pasar por `_()`. Dos advertencias conocidas del repositorio:

- Nunca interpolar dentro de `_()`: `_("Liquidación %s") % run.name`, no `_(f"Liquidación {run.name}")`.
- No evaluar `_()` dentro de una expresión generadora; Odoo resuelve el idioma por *frame* y devuelve el texto en inglés. Las listas de encabezados se construyen con un bucle explícito.

El archivo `i18n/es_D0.po` se genera con `tools/i18n-generator/generate-po.sh` (exportación desde Odoo) y solo se traducen los `msgstr`; no se escribe el `.po` a mano.

11. Pruebas

Automatizadas (`tests/test_landed_cost_run_report.py`, `TransactionCase`):

1. Generar el XLSX de una liquidación validada y verificar que el binario no está vacío y abre con `openpyxl`.
2. Verificar el número y nombre de las hojas, y que la hoja de resumen tiene una fila por producto de `summary_ids`.
3. Verificar los totales del pie contra `fob_currency`, `additional_cost_total_currency`, `amount_total_currency`.
4. Con un usuario **sin** `account.group_account_invoice`, verificar que la hoja de facturas de proveedor no existe.
5. Con un usuario **sin** `group_preclearance`, verificar que la hoja de pre-despacho no existe.
6. Renderizar el PDF con `_render_qweb_pdf` sobre una liquidación con las cuatro secciones pobladas y sobre otra con secciones vacías.
7. Selección múltiple: dos liquidaciones en un mismo libro, sin colisión de nombres de hoja.

Manuales:

- Liquidación en los tres estados (borrador, validada, cancelada).
- Liquidación con productos de pre-despacho con diferencia y asiento generado.
- Liquidación con muchas líneas (>200 productos en el resumen), para verificar la paginación del PDF y el rendimiento del XLSX.
- Impresión del PDF en A4 real, verificando que ninguna tabla se corte a lo ancho.

Para renderizar reportes desde `odoo shell` hay que usar `ir.actions.report._render_qweb_html`; `ir.qweb._render` falla con `KeyError: is_html_empty`.

12. Consideraciones y riesgos técnicos

Punto	Detalle
<code>xlsxwriter</code> en el contenedor	Es requisito de <code>report_xlsx</code> . Verificar que está en el <code>requirements.txt</code> de la imagen antes de instalar el módulo en staging.
Submódulo OCA en el despliegue	<code>OCA/reporting-engine</code> debe estar en la ruta de addons del cliente. Hoy está en el repositorio en rama 19.0; hay que confirmar que el despliegue del cliente lo monta.
Acoplamiento <code>features ↔ file</code>	<code>stock_landed_costs_features</code> usa <code>lc_file_ids</code> sin declararlo. No lo introduce este desarrollo, pero condiciona la dependencia elegida (ver 4.3).
Volumen de líneas	El resumen se reconstruye entero en <code>_update_summary_records()</code> ; en liquidaciones grandes la lectura de <code>previous_cost_factor_currency</code> hace una consulta SQL por fila. Si el reporte se percibe lento, se precalcula el factor anterior en una sola consulta agrupada dentro del generador, en vez de leer el campo calculado fila a fila.
Widget <code>cost_factor_indicator_widget</code>	No es imprimible ni exportable; se sustituye por columnas «Factor anterior» y «Variación» con formato condicional. Es un cambio de presentación que conviene validar con el usuario final.
Columnas opcionales	El pedido es «todos los campos». El reporte fija el conjunto de columnas en código; si el usuario oculta columnas en pantalla, el archivo igual las trae. Alternativa, si se quiere respetar la selección del usuario: leer <code>optional</code> desde el almacenamiento local del cliente web, lo que exige una capa JS y no se recomienda.
Migración a v19	El cliente aún no está en 19. El módulo se desarrolla y prueba sobre 19.0; si se necesita antes en 17.0, el mismo código es portable salvo el nombre de la etiqueta de vista (<code>list</code> vs <code>tree</code>) y la API de grupos, que no afectan a un módulo de solo reportes.

13. Resumen de decisiones

- Módulo nuevo** `stock_landed_costs_run_report`, dependiente de `stock_landed_costs_file` y `report_xlsx`. No se modifica `stock_landed_costs_features`.
- XLSX espejo multi-hoja**: una hoja de cabecera con totales y una hoja por cada una de las cuatro pestañas, con encabezados formateados, importes como números con máscara de moneda, filas de suma como fórmula, autofiltro y paneles congelados.

3. **PDF QWeb apaisado** con formato de papel propio, secciones independientes por pestaña y encabezados de tabla repetidos entre páginas.
4. Ambos reportes se enganchan al menú **Imprimir** del formulario y de la lista de liquidaciones.
5. Los permisos de las pestañas (`account.group_account_invoice` , `group_preclearance`) se replican en el generador; no se heredan de la vista.